

Python 파일 경로 완전 가이드

목차

- 1. 기본 개념
- 2. 상대경로 vs 절대경로
- 3. CWD 이해하기
- 4. Python에서 경로 다루기
- 5. Best Practices
- 6. 실제 사례

기본 개념

경로 기호 정리

기호	의미	예시	설명
.	현재 디렉토리	./file.txt	현재 폴더의 file.txt
..	상위 디렉토리	../file.txt	한 단계 위 폴더의 file.txt
../..	2단계 상위	../../file.txt	두 단계 위 폴더의 file.txt
/	경로 구분자	folder/file.txt	폴더와 파일 구분
~	홈 디렉토리	~/Documents/file.txt	사용자 홈 디렉토리 (Unix/Linux/Mac)

상대경로 vs 절대경로

1. 상대경로 (Relative Path)

정의: 현재 작업 디렉토리(CWD)를 기준으로 파일 위치 지정

```
# 상대경로 예시
./data/trade_history.csv
data/trade_history.csv # ./ 생략 가능
../utils/script.py
../../config/settings.json
```

장점

- 짧고 간결함
- 폴더 이동 시 코드 변경 불필요

단점

- **✗ CWD에 의존** (어디서 실행하는지에 따라 결과 달라짐)

- ## 2. 절대경로 (Absolute Path)

```
# Windows 절대경로
C:\Users\jh700\projects\StockTrader\data\trade_history.csv
E:\dev\StockTrader\utils\script.py

# Unix/Linux/Mac 절대경로
/home/user/projects/data/file.csv
/Users/user/documents/settings.json
```

- ☒ 명확함 (어디서든 같은 파일 참조)
- ☒ CWD 독립적
- ☒ 재현성 높음

- 경로가 길고 복잡함
- 다른 사용자/머신으로 이동 시 수정 필요

```
(.venv) (.venv) E:\dev\StockTrader> python script.py
↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑↑
0 | 부분이 | CWD
```

```
# Windows PowerShell
cd E:\dev\StockTrader
cd D:\projects

# Unix/Linux/Mac
```

```
cd /home/user/projects
cd ~/Documents
```

2. 확인 방법

```
# PowerShell
pwd

# Python 코드
import os
print(os.getcwd()) # 출력: E:\dev\StockTrader
```

3. 프로그램 실행 시에 따라 달라짐

```
# 상황 1: E:\dev\StockTrader 에서 실행
E:\dev\StockTrader> python utils/script.py
# → CWD = E:\dev\StockTrader

# 상황 2: E:\ 에서 실행
E:\> python dev/StockTrader/utils/script.py
# → CWD = E:\
```

⚠ CWD 의존의 위험성

```
# remove_ks.py - 위험한 코드 (CWD 의존)
import pandas as pd

df = pd.read_csv('./data/trade_history.csv')
# CWD가 E:\dev\StockTrader이면 ☑ 성공
# CWD가 E:\이면 ✕ FileNotFoundError
```

Python에서 경로 다루기

주요 함수

1. `os.getcwd()` - 현재 CWD 조회

```
import os

cwd = os.getcwd()
print(cwd)
# 출력: E:\dev\StockTrader
```

2. `__file__` - 현재 스크립트의 경로

```
print(__file__)  
# 출력: utils/jupyter_notebook/remove_ks.py  
# 또는: E:\dev\StockTrader\utils\jupyter_notebook\remove_ks.py
```

중요: `__file__`은 상대경로일 수도, 절대경로일 수도 있습니다.

3. `os.path.abspath()` - 상대경로 → 절대경로 변환

```
import os  
  
relative = "./data/file.csv"  
absolute = os.path.abspath(relative)  
print(absolute)  
# 출력: E:\dev\StockTrader\data\file.csv
```

4. `os.path.dirname()` - 디렉토리 추출

```
import os  
  
path = "E:\dev\StockTrader\utils\script.py"  
directory = os.path.dirname(path)  
print(directory)  
# 출력: E:\dev\StockTrader\utils
```

5. `os.path.join()` - 경로 결합

```
import os  
  
dir_path = "E:\dev\StockTrader"  
file_name = "data\file.csv"  
full_path = os.path.join(dir_path, file_name)  
print(full_path)  
# 출력: E:\dev\StockTrader\data\file.csv
```

장점: OS별 경로 구분자 자동 처리 (\ vs /)

6. `os.path.normpath()` - 경로 정규화

```
import os

messy_path = "E:\\dev\\..\\dev\\StockTrader\\data\\file.csv"
clean_path = os.path.normpath(messy_path)
print(clean_path)
# 출력: E:\\dev\\StockTrader\\data\\file.csv
```

7. pathlib.Path - 모던 방식 (권장)

```
from pathlib import Path

# 절대경로 생성
script_file = Path(__file__).resolve() # 절대경로
script_dir = script_file.parent # 부모 디렉토리

# 상위 디렉토리로 이동
data_dir = script_dir.parent.parent / "data" # ../와 같음

# CSV 읽기
csv_file = data_dir / "trade_history.csv"
print(csv_file)
# 출력: E:\\dev\\StockTrader\\data\\trade_history.csv
```

Best Practices

☒ Rule 1: 스크립트 위치 기반 경로 사용

스크립트 자신의 위치를 기준으로 경로를 설정하세요.

```
import os
import pandas as pd

# ☒ 좋음: __file__을 절대경로로 변환
script_dir = os.path.dirname(os.path.abspath(__file__))
csv_path = os.path.join(script_dir, "../..\\data\\trade_history.csv")
csv_path = os.path.normpath(csv_path)

df = pd.read_csv(csv_path)
```

☒ Rule 2: 경로 정규화하기

상위 디렉토리 이동(..) 후에는 반드시 `normpath()` 사용:

```
import os

# ✕ 피해야 할 것
path = os.path.join(script_dir, "../data/file.csv")

# ☑ 권장
path = os.path.normpath(os.path.join(script_dir, "../data/file.csv"))
```

☑ Rule 3: pathlib 사용 (Python 3.4+)

더 깔끔하고 안전합니다:

```
from pathlib import Path
import pandas as pd

# 현재 스크립트 디렉토리
script_dir = Path(__file__).resolve().parent

# 상위 2단계 이동 후 data 폴더
csv_path = script_dir.parent.parent / "data" / "trade_history.csv"

# CSV 읽기
df = pd.read_csv(csv_path)
```

☑ Rule 4: 절대경로 확인

디버깅 시 항상 절대경로 출력:

```
import os

csv_path = "./data/trade_history.csv"

# ✕ 상대경로 출력 (어디를 가리키는지 불명확)
print(f"파일: {csv_path}")

# ☑ 절대경로 출력 (정확하게 표시)
abs_path = os.path.abspath(csv_path)
print(f"파일: {abs_path}")
```

☑ Rule 5: 파일 존재 여부 확인

```
import os

csv_path = "./data/trade_history.csv"

if not os.path.exists(csv_path):
```

```

    raise FileNotFoundError(f"파일을 찾을 수 없습니다:
{os.path.abspath(csv_path)}")

# 처리 계속...

```

☑ Rule 6: 프로젝트 루트 경로 설정

대규모 프로젝트의 경우, 프로젝트 루트를 명시:

```

import os
from pathlib import Path

# 프로젝트 루트 찾기 (가정: 이 파일이 src/utils/script.py)
PROJECT_ROOT = Path(__file__).resolve().parent.parent.parent
DATA_DIR = PROJECT_ROOT / "data"
CONFIG_DIR = PROJECT_ROOT / "config"

print(f"프로젝트 루트: {PROJECT_ROOT}")
print(f"데이터 디렉토리: {DATA_DIR}")

```

실제 사례

사례 1: CSV 파일 처리 (현재 프로젝트)

```

import os
import pandas as pd

# ✕ 위험: CWD 의존
def read_csv_bad():
    df = pd.read_csv('./data/trade_history.csv')
    return df

# ☑ 안전: 절대경로 사용
def read_csv_good():
    script_dir = os.path.dirname(os.path.abspath(__file__))
    csv_path = os.path.join(script_dir, "../../data/trade_history.csv")
    csv_path = os.path.normpath(csv_path)

    if not os.path.exists(csv_path):
        raise FileNotFoundError(f"파일 없음: {csv_path}")

    df = pd.read_csv(csv_path)
    return df

# ☑ 최고: pathlib 사용
def read_csv_best():
    from pathlib import Path

```

```

    csv_path = Path(__file__).resolve().parent.parent.parent / "data" /
    "trade_history.csv"

    if not csv_path.exists():
        raise FileNotFoundError(f"파일 없음: {csv_path}")

    df = pd.read_csv(csv_path)
    return df

```

사례 2: 설정 파일 로드

```

from pathlib import Path
import json

# 프로젝트 루트 기준
PROJECT_ROOT = Path(__file__).resolve().parent.parent
CONFIG_FILE = PROJECT_ROOT / "config" / "settings.json"

def load_config():
    if not CONFIG_FILE.exists():
        print(f"설정 파일 없음: {CONFIG_FILE}")
        return {}

    with open(CONFIG_FILE, 'r') as f:
        config = json.load(f)

    return config

```

사례 3: 로그 파일 저장

```

from pathlib import Path
from datetime import datetime

def save_log(message):
    # 프로젝트 루트/logs 디렉토리
    log_dir = Path(__file__).resolve().parent.parent / "logs"
    log_dir.mkdir(exist_ok=True) # 없으면 생성

    log_file = log_dir / f"log_{datetime.now().strftime('%Y%m%d')}.txt"

    with open(log_file, 'a', encoding='utf-8') as f:
        f.write(f"[{datetime.now()}] {message}\n")

    print(f"로그 저장: {log_file}")

```

트러블슈팅

문제 1: FileNotFoundError

```
# ✕ 오류: FileNotFoundError: [Errno 2] No such file or directory
df = pd.read_csv("../data/file.csv")
```

해결책:

```
# ☑ 해결
import os

script_dir = os.path.dirname(os.path.abspath(__file__))
csv_path = os.path.normpath(os.path.join(script_dir, "../data/file.csv"))
print(f"찾는 경로: {csv_path}") # 디버깅용 출력
df = pd.read_csv(csv_path)
```

문제 2: 다른 팀원 PC에서 작동 안 함

```
# ✕ 위험: 절대경로 하드코딩
csv_path = "C:\\Users\\jh700\\projects\\StockTrader\\data\\file.csv"

# ☑ 해결: __file__을 기준으로
import os
script_dir = os.path.dirname(os.path.abspath(__file__))
csv_path = os.path.join(script_dir, "../data/file.csv")
```

문제 3: IDE vs 터미널에서 경로 다르게 동작

```
# ✕ CWD 의존 (IDE에서는 프로젝트 루트, 터미널에서는 다를 수 있음)
df = pd.read_csv("./data/file.csv")

# ☑ 해결: 항상 절대경로 생성
from pathlib import Path
csv_path = Path(__file__).resolve().parent.parent / "data" / "file.csv"
```

요약 체크리스트

프로젝트를 시작할 때 다음을 확인하세요:

- ☐ 상대경로 대신 `__file__` 기반 절대경로 사용
- ☐ `os.path.normpath()` 또는 `pathlib.Path` 사용
- ☐ 파일 존재 여부 확인 후 처리
- ☐ 절대경로로 디버깅 메시지 출력
- ☐ CWD 의존성 제거

- ☐ 다른 팀원 PC에서 테스트
 - ☐ IDE와 터미널에서 모두 테스트
-

참고 자료

- [Python os.path 문서](#)
- [Python pathlib 문서](#)
- [Windows vs Unix 경로](#)